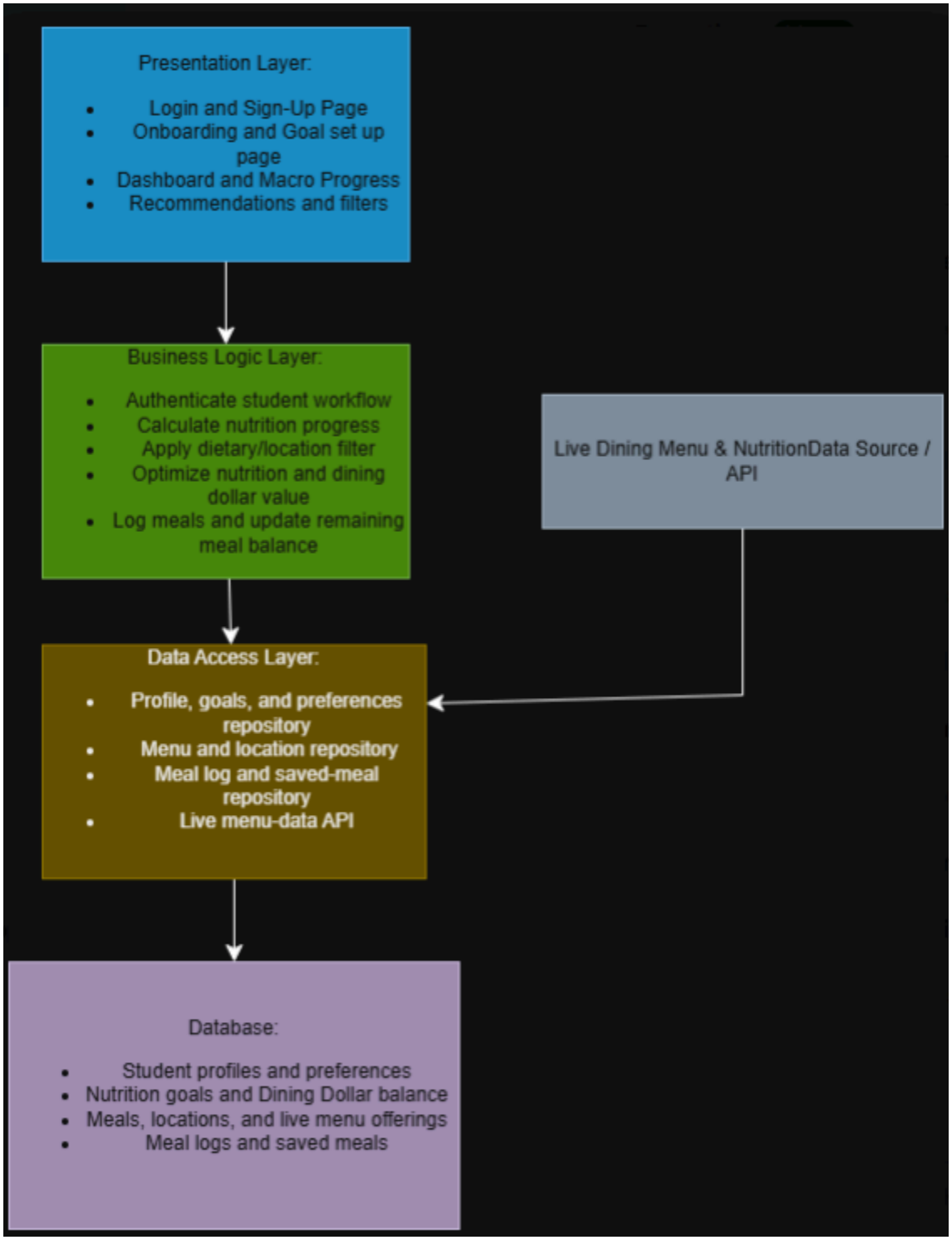
Alan Mathew
Max Plankar
Aarav Babel

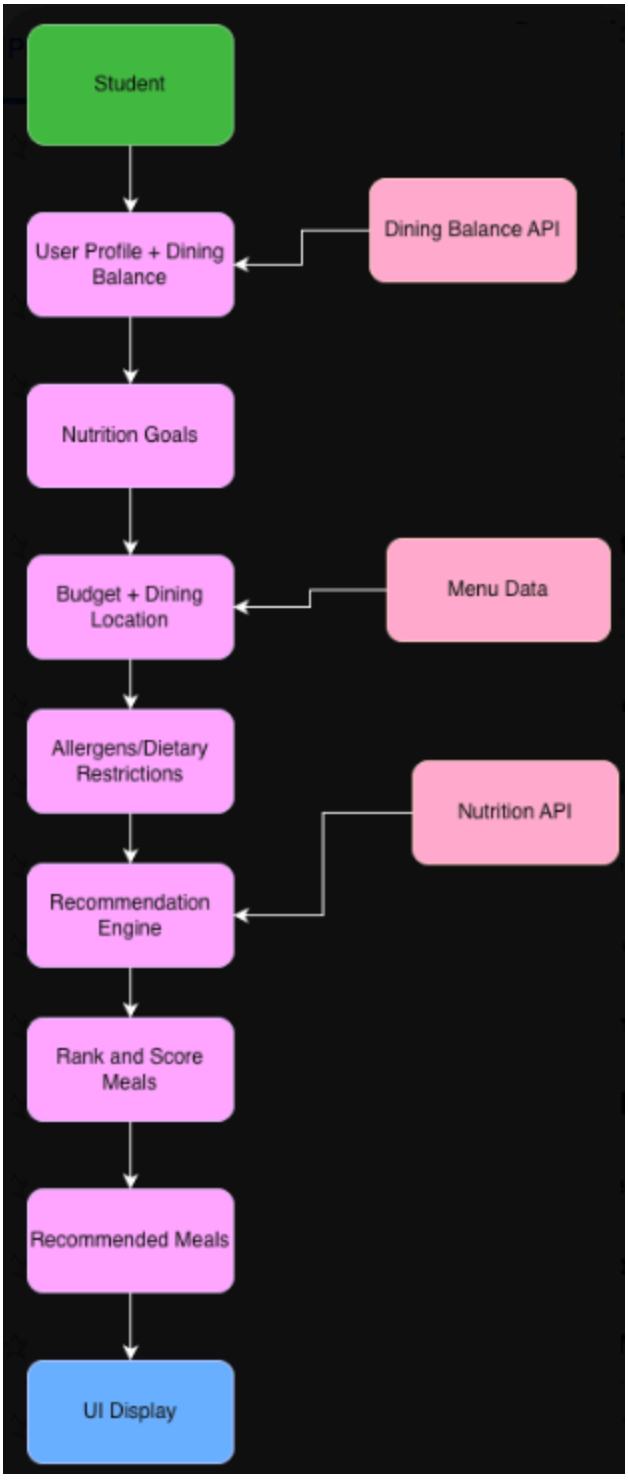
TE4 - Design and Architectural Patterns
Team MANGO

Proposed Architecture:

- **Architecture Pattern:**
  - **Primary:** Layered
  - **Secondary:** Pipe and Filter
- **Brief Explanation:**
  - **Primary:** We chose Layered as our primary as the recommendation algorithm (nutrition fit + Dining Dollar value + preferences) best belongs in one business logic layer instead of being mixed into react pages or database queries. It will also keep the code an organized format that would be easy to follow as features grow.
  - **Secondary:** We chose pipe and filter as our secondary architecture pattern solely for a future feature of taking inconsistent incoming menu data and converting it into clean meals and menu_offerings records.
- **Diagram:**
  - **Layered:**



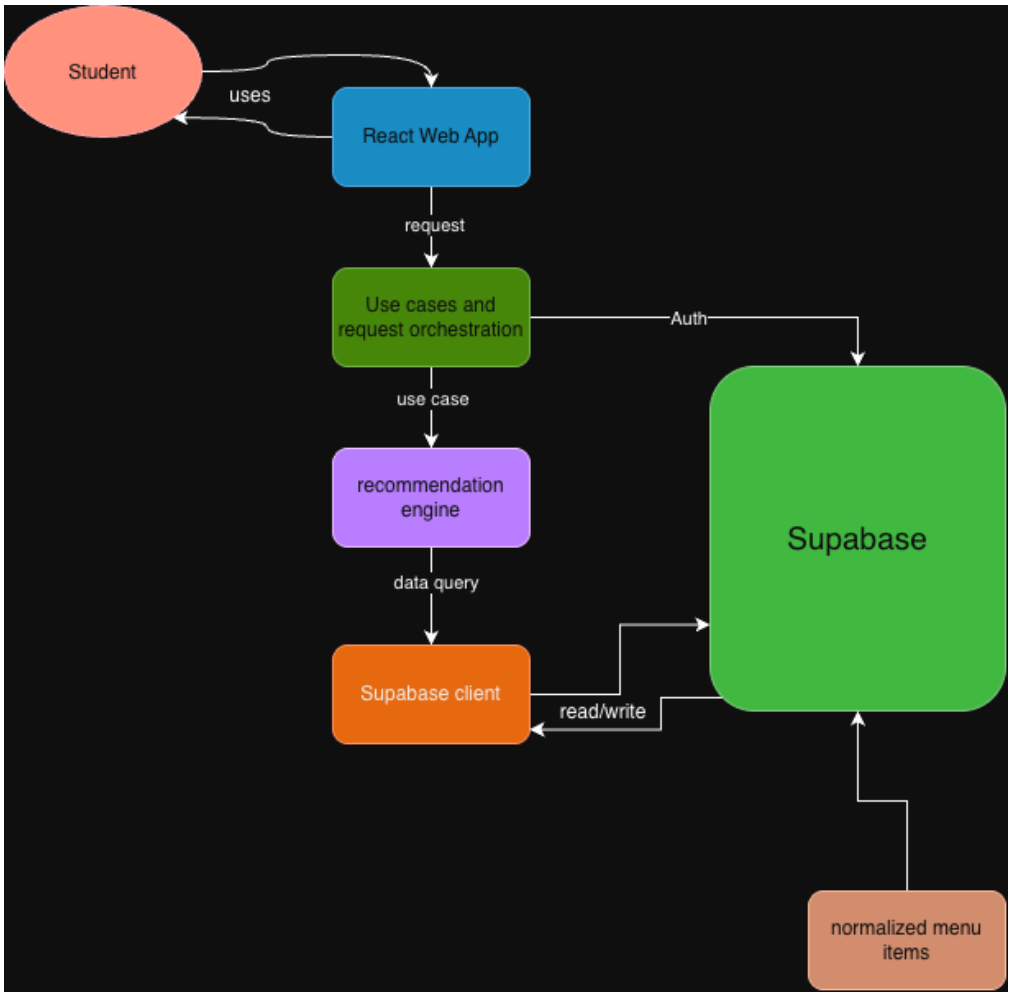
- **Pipe and Filter:**



- **Direction of Communication:**
  - **Request:**
    - **Student -> Presentation -> Business Logic -> Data Access -> Supabase/ live menu source**
  - **Response:**

- **Supabase/ live menu source -> Data Access -> Business Logic -> Presentation -> Student**

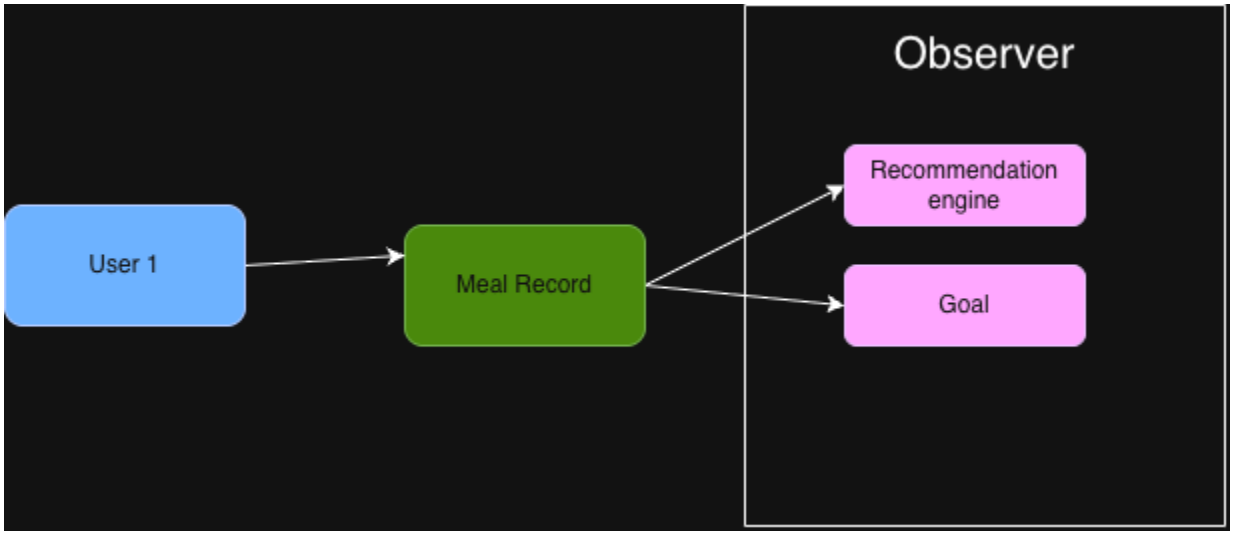
High Level Design Diagram:



Design Patterns (Planned):

- **Name of the Pattern:** Observer
- **Problem it Solves:** The problem it solves is that when the Student logs, edits, or removes data, it updates the several parts of PolyProvision that need the updated information such as the recommendation algorithm and the goals.
- **Identify Modules, Components, and Services:**
  - Modules:
    - Dashboard, Supabase
    - Subject:
      - MealRecord
    - Observers:

        - Recommendation engine
        - Goal
    - Services:
        - New menu items update the menu database
        - Dining Dollar Tracker
- **UML Diagram:**



- **Explain why this Pattern is a better choice than placing all of the logic in one large component or module:** This works great for us, since our app's global menu database is meant to be accessed in some way by all users. The observer design pattern allows our app to easily connect users with the entire campus menu without rate limits and modularly.

Design Decisions and Tradeoffs:

Decision 1: Deterministic, rule-based recommendations instead of an AI/LLM model

Options considered: We discussed using an LLM API to suggest meals versus writing our own ranking logic that filters and scores menu items.

What we chose: For the MVP, recommendations will be deterministic. The system first filters out meals that are unavailable or conflict with a student's dietary restrictions, allergens, or preferred locations. It then ranks what remains by how well each meal fits the student's remaining calorie and macro targets and their Dining Dollar budget, while balancing nutrition against cost rather than just picking the cheapest option. Each recommendation includes a short explanation, like "High protein and within today's Dining Dollar target."

Why it fits: Rule-based scoring gives the same result for the same inputs, which makes it much easier to test and debug. Students can see why a meal was suggested, and we avoid paying for an outside AI service or sending any student data to one. The tradeoff is that our rules will be

less flexible than a learned model, but our menu data set (the items we collected from Taco Bell, Panda Express, Subway, and Chick-fil-A) is small enough that clear rules work well for now.

Decision 2: Building a mobile-first web app instead of a native mobile app

Options considered: Since most students would use PolyProvision on their phones, we considered building a native mobile app with React Native (Expo) versus a web app with React, TypeScript, and Vite that is designed for phone screens.

What we chose: We built a mobile-first web app in apps/web. The default styles are written for phones: screens take up the full width, images scale to fit, and the layout supports widths as small as 320 pixels. Navigation uses a bottom nav bar, which is a pattern students already know from phone apps. On larger screens, a min-width: 768px media query keeps the app at a phone-sized 390 by 844 pixel frame centered in the browser, so desktop users see the same layout as phone users.

Why it fits: Our team already knows React, so we can spend our time on the core features (onboarding, the dashboard, and meal recommendations) instead of learning React Native during the quarter. A web app also runs on any phone through the browser, so we avoid app store setup and separate iOS and Android testing, and every teammate can test changes right away with npm run dev. Keeping one phone-sized layout on every device also means we only have to design and test one version of each screen. The tradeoff is that we give up native features like push notifications and a downloadable app, and desktop users don’t get a layout that uses their extra screen space. Since students will mostly check PolyProvision on their phones before getting food, we decided this was acceptable for the MVP. We also left room to change course later: a separate Expo client could live in apps/mobile and reuse our shared TypeScript types, Express API, and Supabase backend, with only the UI components and styles needing to be rewritten.